What's new in .NET MAUI for .NET 11

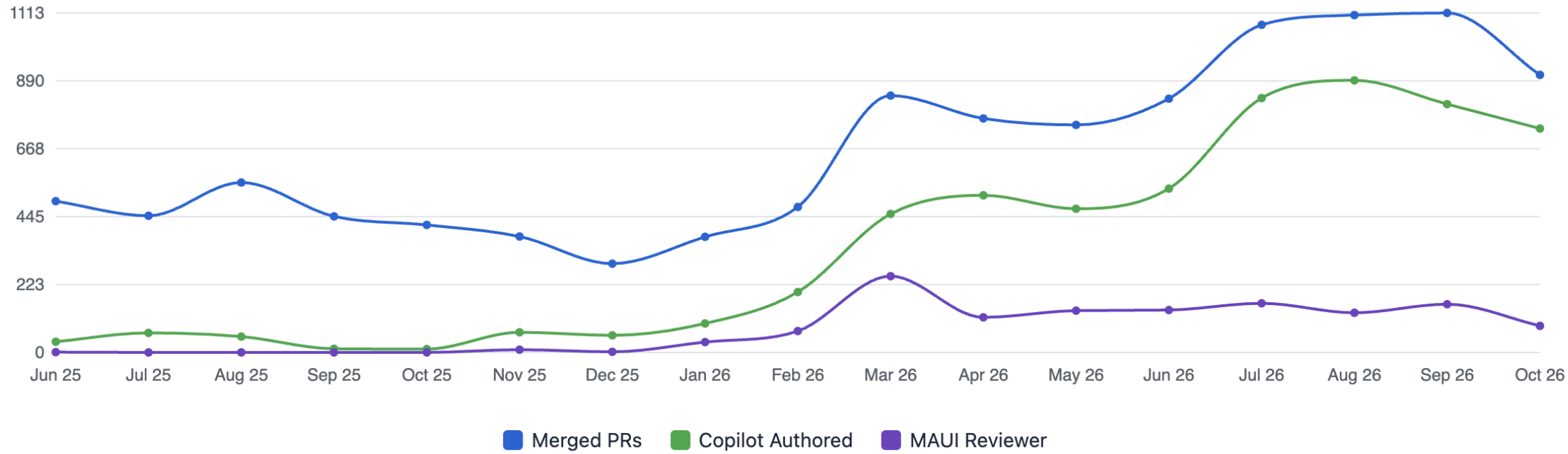


Gerald Versluis

Software Engineer, Microsoft

Total Merged PRs	Copilot Authored	MAUI Reviewer Used
11282	5840	1293

Monthly Totals

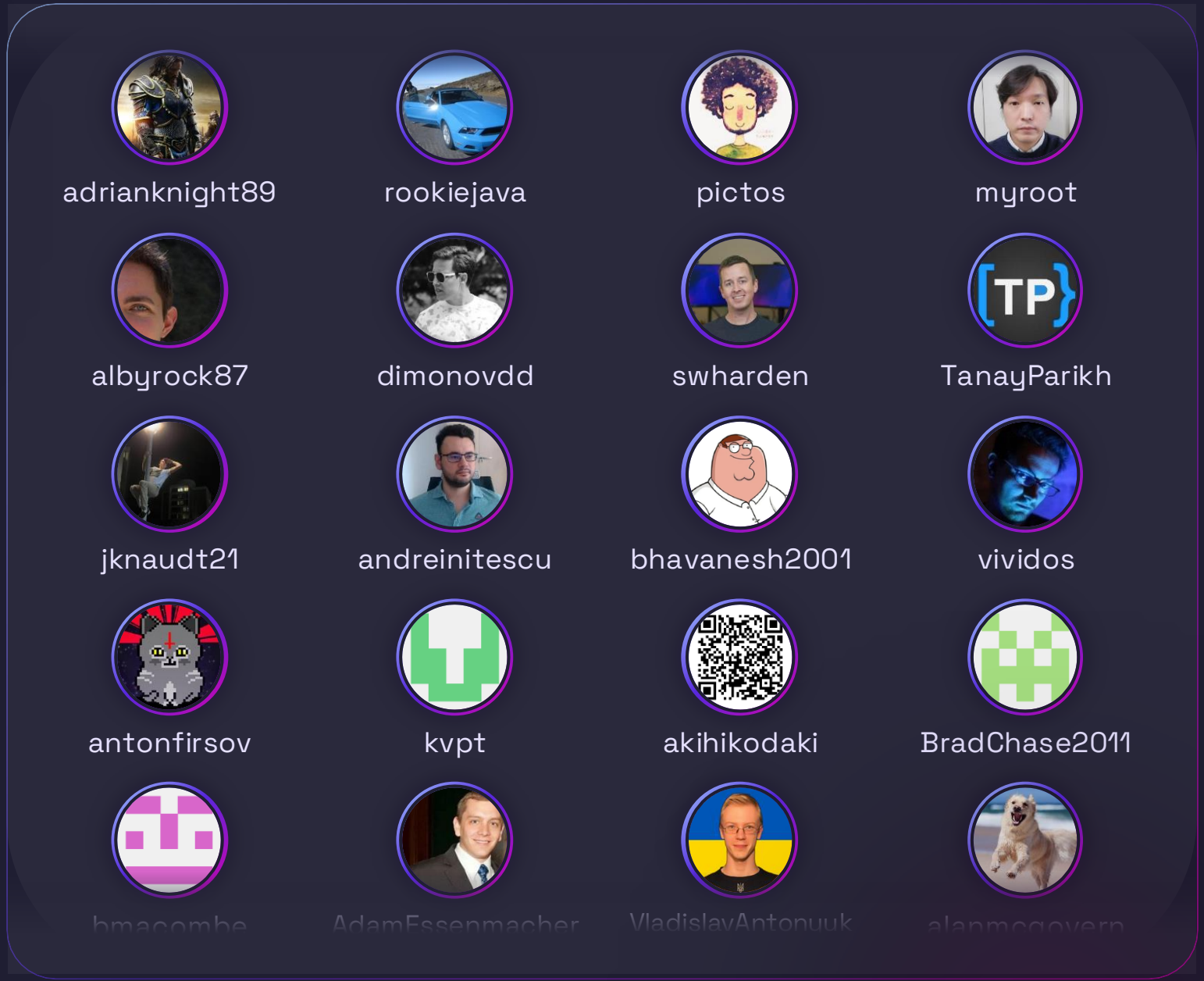


Source: agentic-engineers.net

github.com/dotnet/maui

Thank you, community contributors!

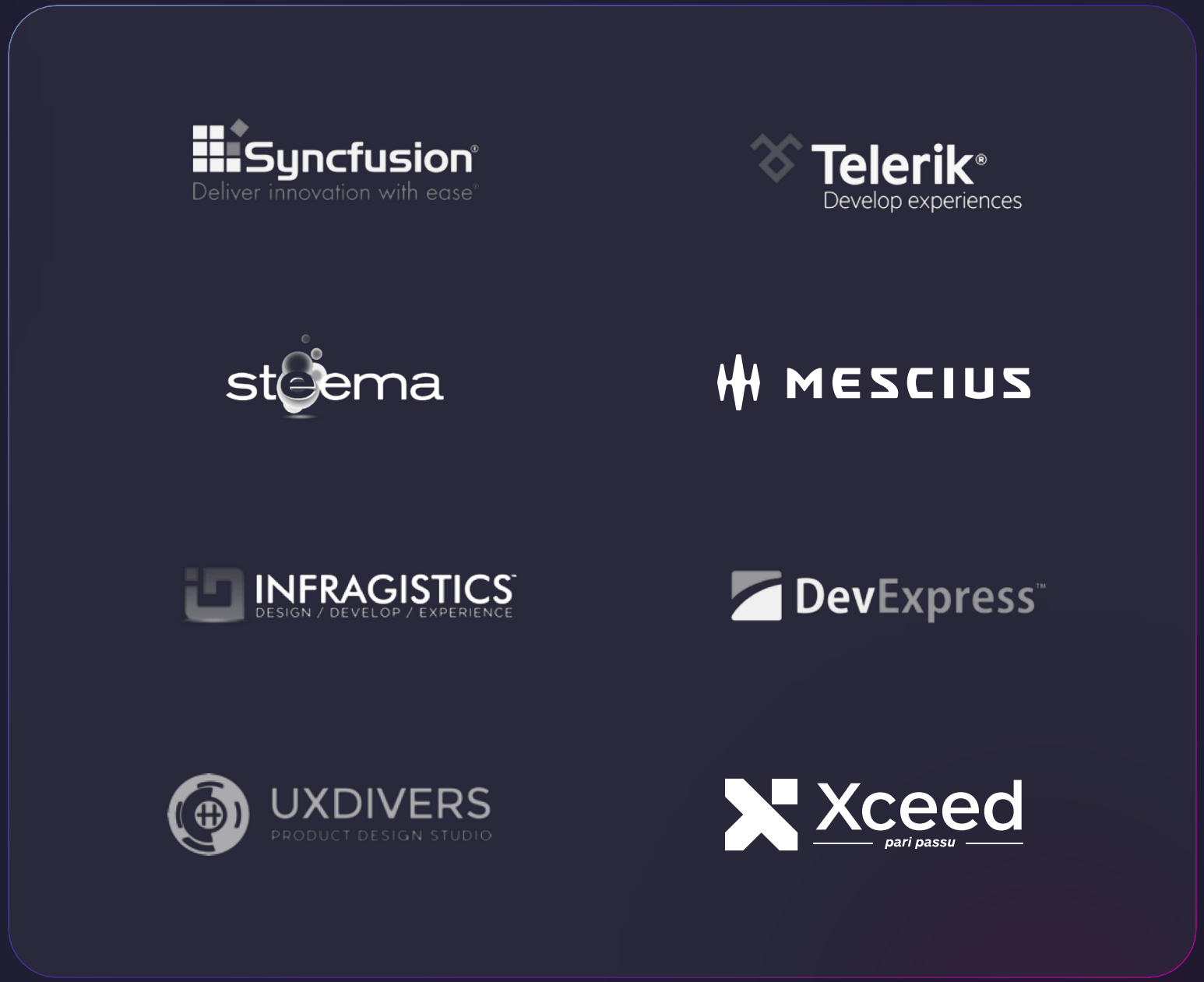
Every pull request, issue and review makes .NET MAUI better for everyone.



.NET MAUI ecosystem

Thank you, component vendors!

Their control suites help .NET MAUI developers ship polished apps faster.

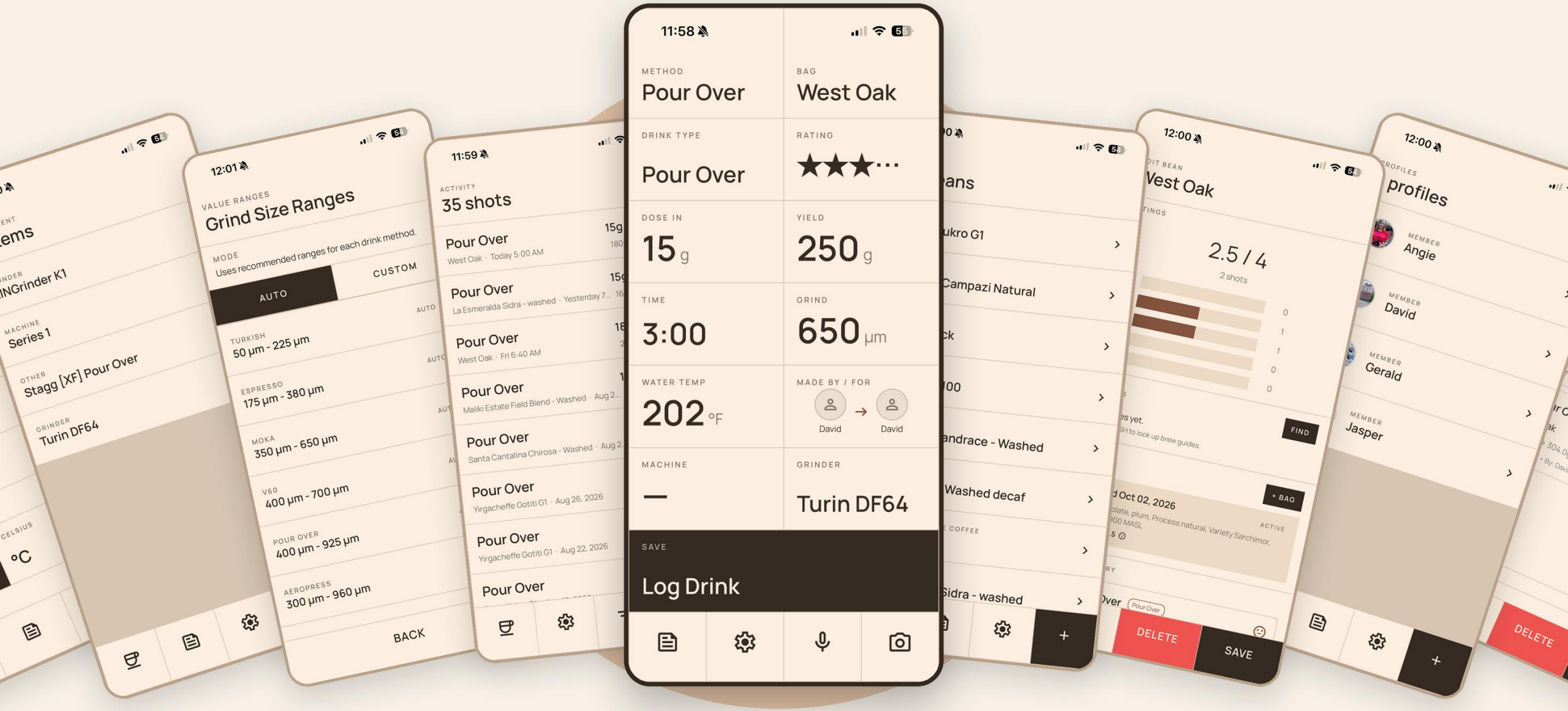


01 CoreCLR



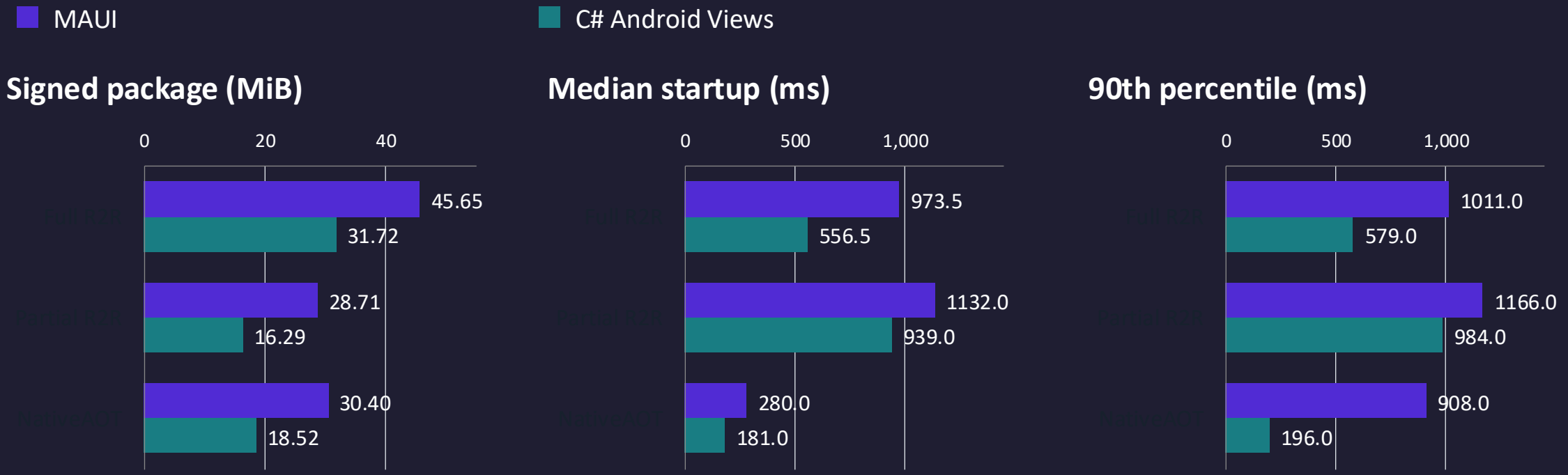
The biggest change to .NET MAUI you won't see, but you will feel.

Barista Notes



BaristaNotes: Android package size and startup

Pixel 5, Android 14 | .NET 11 RC2 | 1,000 drinks | 30 launches per cell



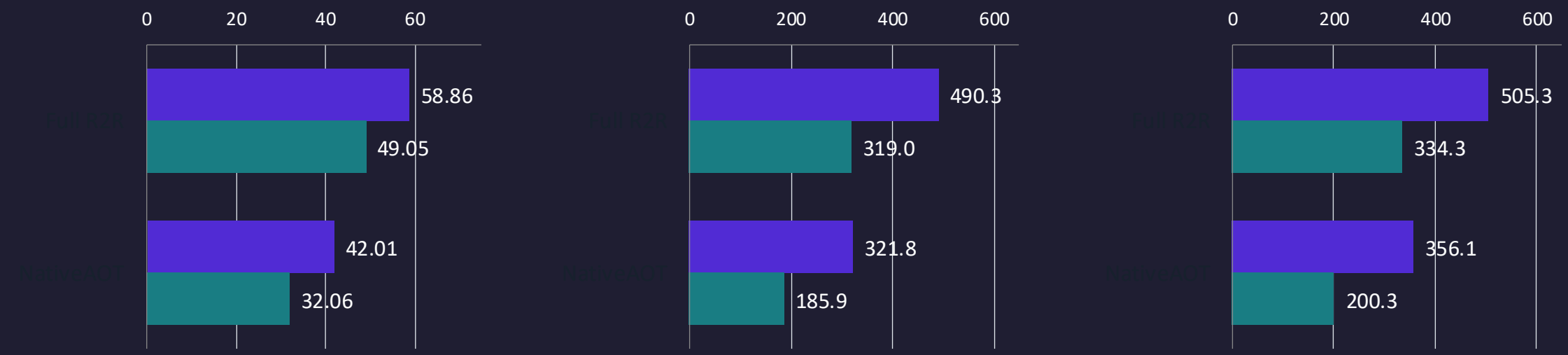
Each panel has its own scale; zero baseline throughout.

Process-cold startup uses Android TotalTime. Signed benchmark packages include the fixture and probes. Android NativeAOT is experimental.

BaristaNotes: iOS package size and startup

iPhone 15 Pro, iOS 26.7.1 | .NET 11 RC2 | 1,000 drinks | 30 launches per cell

MAUI C# UIKit



Each panel has its own scale; zero baseline throughout.

Startup: kernel process start to New Drink readiness plus two frames. MAUI NativeAOT has an accepted compiler warning; measured routes only.

02 XAML improvements



20 years of XAML, from WPF to .NET MAUI. Familiar syntax, brand-new tricks.

Global XMLNS

```
// XAML namespace declarations in root and node prefixes
<ContentPage xmlns=
  "http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="MyApp.MainPage"
  xmlns:vm="clr-namespace:MyApp.ViewModels"
  xmlns:conv="clr-namespace:MyApp.Converters">
  <vm:MainViewModel />
</ContentPage>

// Add your own namespaces to the global xmlns, once per assembly
[assembly: XmlnsDefinition("http://schemas.microsoft.com/dotnet/2021/maui/global",
                           "clr-namespace:MyApp.ViewModels;assembly=MyApp")]
[assembly: XmlnsDefinition("http://schemas.microsoft.com/dotnet/2021/maui/global",
                           "clr-namespace:MyApp.Converters;assembly=MyApp")]
```

Implicit XMLNS

```
// No more need for xmlns declarations
<ContentPage xmlns=
  "http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="MyApp.MainPage">
  <MainViewModel />
</ContentPage>

// Implicit by default in .NET 11. On .NET 10, opt in from your .csproj:
<PropertyGroup>
  <DefineConstants>$(DefineConstants);
    MauiAllowImplicitXmlnsDeclaration</DefineConstants>
  <EnablePreviewFeatures>true</EnablePreviewFeatures>
</PropertyGroup>
```

XmlnsDefinition vs. XmlnsPrefix

```
// No xmlns, no xmlns:x, no xmlns:mylib
<ContentPage x:Class="MyApp.MainPage">
  <FancyButton />        <!-- A: no prefix -->
  <mylib:FancyButton />  <!-- B: prefix, nothing to declare -->
</ContentPage>

// A. Map the library into the global xmlns
[assembly: XmlnsDefinition("http://schemas.microsoft.com/dotnet/maui/global",
                           "http://mylib.com/xaml")]

// B. Give it a global prefix, handy when names clash
[assembly: XmlnsPrefix("http://mylib.com/xaml", "mylib")]
```

Before: an InverseBoolConverter

```
// 1. Write a converter class
public class InverseBoolConverter : IValueConverter
{
    public object Convert(object value, Type targetType,
        object parameter, CultureInfo culture) => !(bool)value;

    public object ConvertBack(object value, Type targetType,
        object parameter, CultureInfo culture) => !(bool)value;
}

// 2. Register it as a resource
<ContentPage.Resources>
  <local:InverseBoolConverter x:Key="InverseBool" />
</ContentPage.Resources>

// 3. Reference it in every binding
<Button IsEnabled="{Binding IsBusy, Converter={StaticResource InverseBool}}" />
```

After: C# expressions

```
// No converter class. No resource. No StaticResource.
<Button IsEnabled="{!IsBusy}" />                // InverseBoolConverter

// The same one-liners replace most other converters
<Label Text="{$'Hello {Name}!'}" />             // StringFormat
<Label Text="{Price * Quantity}" />             // MultiplyConverter
<Label Text="{Title ?? 'Untitled'}" />          // TargetNullValue
<Label IsVisible="{IsAdmin || IsOwner}" />      // MultiBinding + converter
<Button Clicked="{(s, e) ⇒ Count++}" />         // EventToCommandBehavior

// Compiled by XAML source generation, typed through x:DataType
```

03 New features & bugfixes



A lot more in .NET MAUI 11

Long press gesture
New recognizer for press-and-hold actions

Tab & toolbar badges
BadgeText on tabs and toolbar items

Passkeys
Face ID, Windows Hello and Android biometrics

Material 3
Android controls follow Material 3 design

Maps
Pin clustering, custom pin images, JSON styles

3,500+ bugs fixed in .NET 11
Estimate: ~40% of 8,900 merged PRs

Animation control
Cancel one animation at a time

BoxView gradients
Fill accepts any brush, including gradients

C# UI template
Start a new app in C# with --ui csharp

System bar colors
Status and nav bars match your app

Smaller apps
Unused CSS support is trimmed at publish

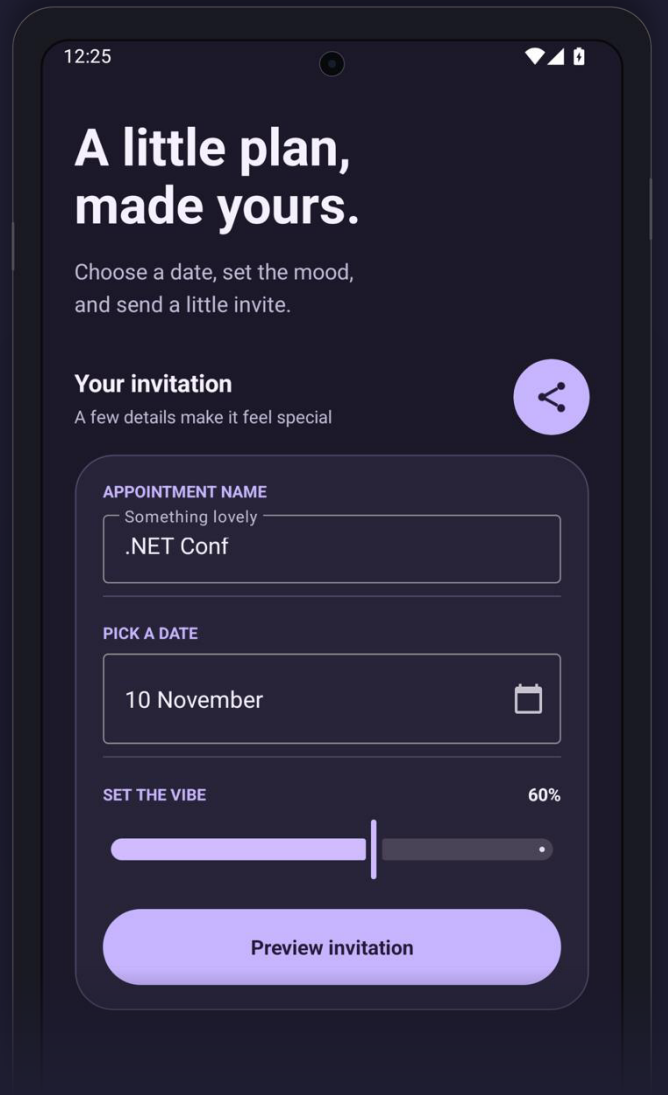
Material 3

One build property modernizes your Android controls

```
<!-- Opt in, in your .csproj -->
<PropertyGroup>
  <UseMaterial3>true</UseMaterial3>
</PropertyGroup>

<!-- Your XAML stays the same -->
<Entry Placeholder="Email" />
<DatePicker />
<Slider Value="0.6" />

// .NET 11: Material 3 types are public
// for custom handlers
class MyEntry : MauiMaterialEditText
```



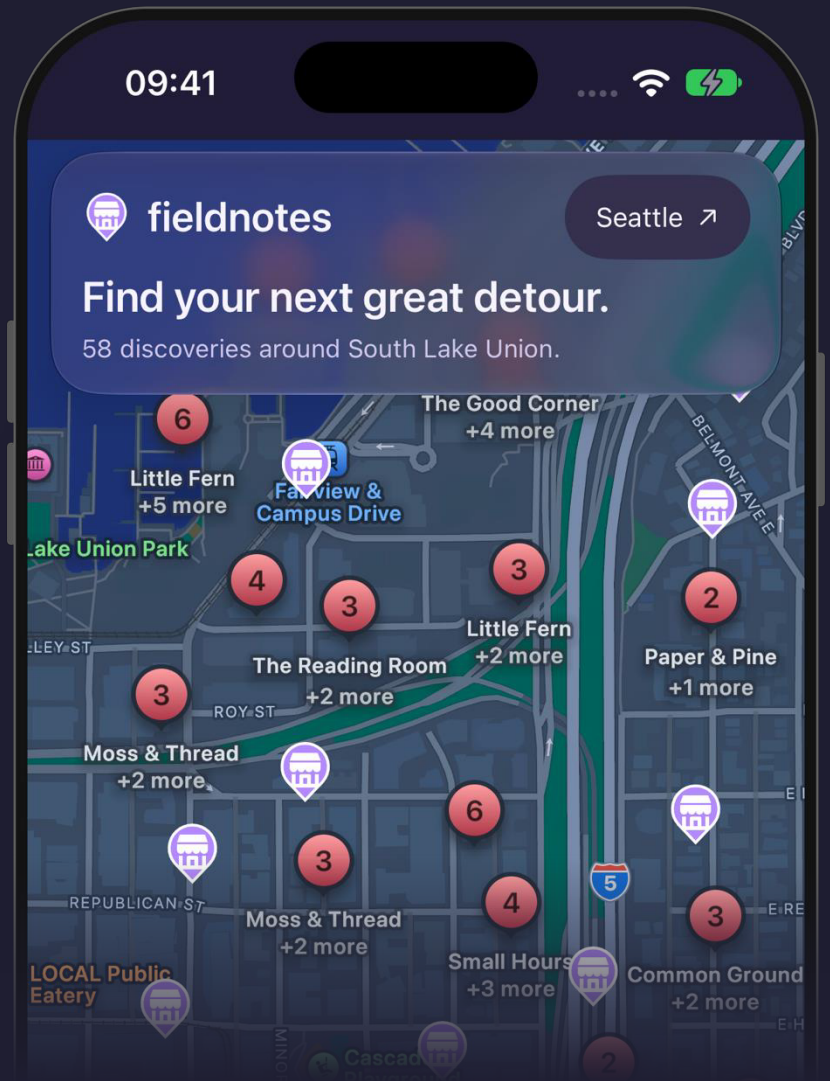
Maps

Hundreds of pins, still one readable map

```
<maps:Map IsClusteringEnabled="True"
          ClusterClicked="OnClusterClicked">

  <maps:Pin Label="Seattle"
            Location="47.6062,-122.3321"
            ClusteringIdentifier="stores"
            ImageSource="store.png" />

</maps:Map>
```



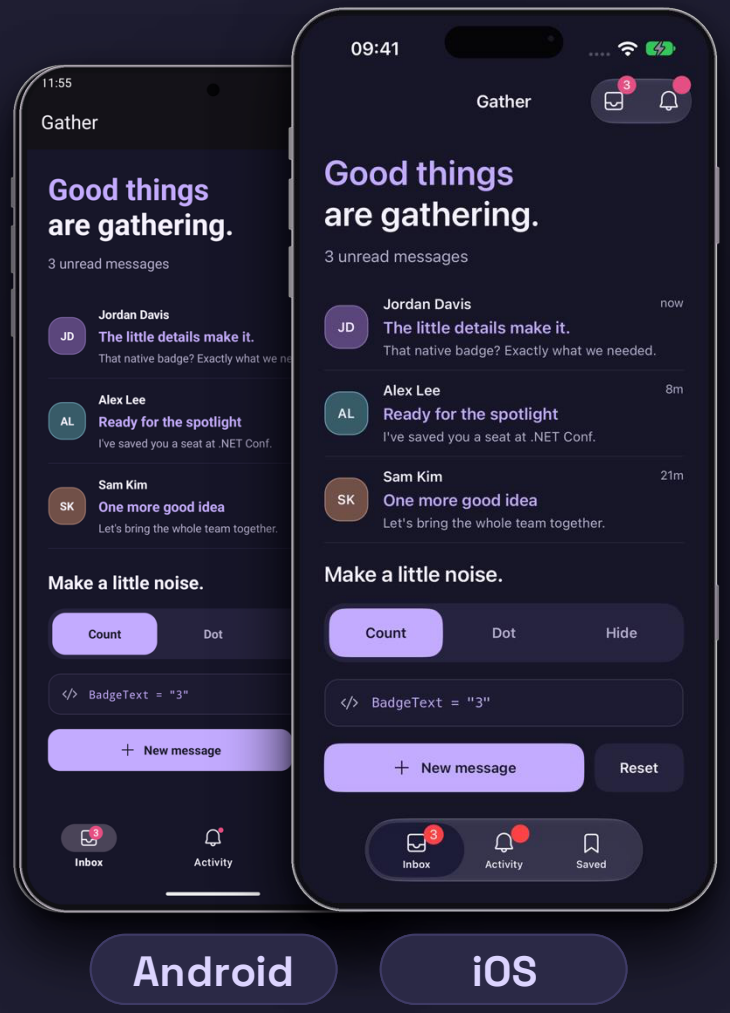
Badges on tabs and toolbar items

Counts and status dots with one property

```
// Toolbar item, bound to the view model
<ToolbarItem Text="Inbox"
  BadgeText="{Binding UnreadCount}" />

// Tab on a TabbedPage
<ContentPage Title="Inbox"
  TabbedPage.BadgeText="3"
  TabbedPage.BadgeColor="Red" />

// "" shows a dot, null hides the badge
```



Passkeys

Sign in with Face ID, Windows Hello or a fingerprint

```
using Microsoft.Maui.Authentication;

if (Passkeys.IsSupported)
{
    // Standard WebAuthn JSON in and out
    var response = await Passkeys
        .AssertAsync(optionsJson);

    await server.VerifyAsync(response);
}
```

Screenshot goes here

Native passkey prompt in a .NET MAUI app

04 Tooling



dotnet run

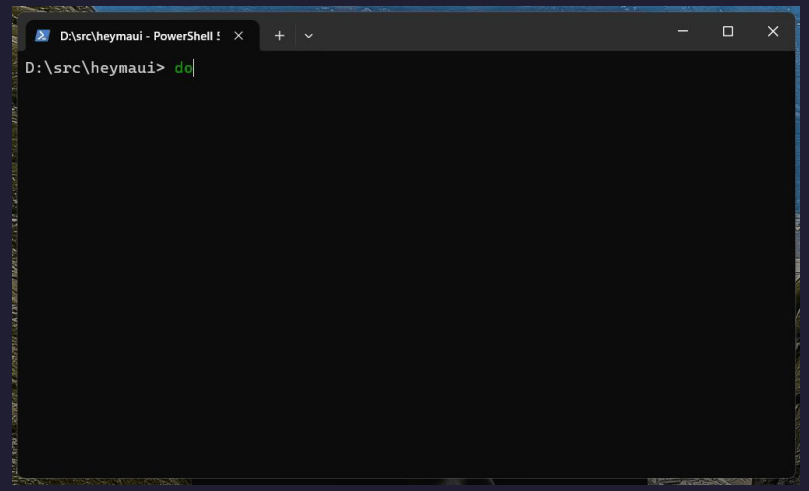
Pick a target and a device right in the terminal

```
# Multi-targeted MAUI project? Just run it
dotnet run

# Prompts for a target framework:
#   net11.0-android, net11.0-ios, ...
# then for a device, emulator or simulator

# App output streams to the terminal;
# Ctrl+C stops the app

# Skip the prompts in scripts
dotnet run -f net11.0-android
```





dotnet test

The same test workflow, now on devices and simulators

```
# New MSTest templates per platform
dotnet new androidtest -n MyTests
# also: iostest, macostest, maccatalysttest

cd MyTests
dotnet test

# Tests run in an app process on the device,
# emulator or simulator you pick

# Run-level controls
dotnet test --timeout 90s
```



dotnet watch

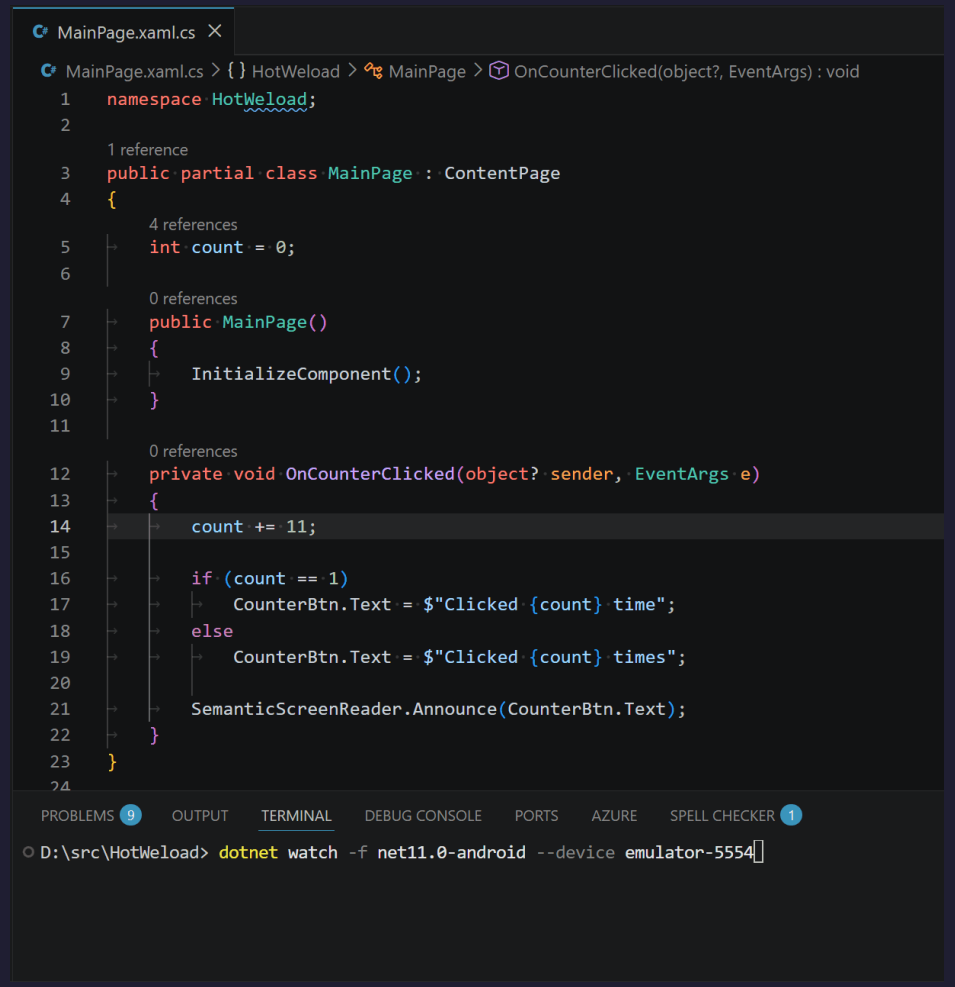
Hot Reload from the terminal, now on Android and iOS

```
# Start the app with Hot Reload
dotnet watch
# same prompts as dotnet run

# Edit C# or XAML and save:
# changes apply to the running app

# Change Hot Reload can't apply?
# press Ctrl+R to restart the app

# Go straight to a platform
dotnet watch -f net11.0-ios
```



XAML Incremental Hot Reload

Edit XAML, keep your place in the app

```
# On by default for Debug builds
# Edits patch every live page, in place:
#   properties and bindings
#   resources and DynamicResource
#   add, remove or reorder children

# CollectionView cells update without
# losing selection or scroll position

# Works in Visual Studio and dotnet watch
dotnet watch
```

GIF goes here

A XAML edit landing on a page that's already open

05 MAUI Labs & AI



Experiments to try in .NET MAUI Labs

MAUI CLI
Setup, SDKs and emulators

DevFlow
Inspect, automate and debug running apps

Essentials.AI
On-device chat and embeddings

AI Extensions
Turn methods into AI tools

Agent skills
maui ai init sets up skills and MCP

10+

experimental projects and more in the repo

Explore it all at github.com/dotnet/maui-labs

Comet & Go
MVU UI in C# and single-file prototypes

Linux backend
MAUI apps on Linux via GTK

WPF backend
22+ controls, Shell and Blazor on WPF

MAUI Testing
dotnet new mauitest for device tests

AppProjectRef
Consume .apk, .ipa and .msix in MSBuild

Four Labs tools worth trying first

MAUI CLI

One tool for setup and devices

- **maui doctor** finds and fixes environment issues
- Installs Android SDK, JDK and emulators
- Manages Xcode, simulators and runtimes
- **--json** output for CI pipelines

```
maui doctor
```

DevFlow

See and drive your running app

- Visual tree, taps and screenshots via an in-app agent
- Network monitoring and profiling
- Blazor WebViews over Chrome DevTools
- MCP server lets AI agents test your app

```
maui devflow mcp
```

Essentials.AI

AI that runs on the device

- Chat and embeddings through Microsoft.Extensions.AI
- Apple Intelligence and NaturalLanguage underneath
- Streaming, structured output and tool calling
- Image input on supported systems

```
Microsoft.Extensions.AI
```

Agent skills

Set up your AI agent in one step

- Installs MAUI and DevFlow skills
- Adds Copilot agents and MCP config
- Targets VS Code, Copilot CLI and Claude
- **--dry-run** previews every change

```
maui ai init
```

MAUI CLI

maui doctor checks your whole setup and fixes what it can.

Android SDKs, JDKs, emulators, Xcode and simulators are managed from one place.

--json and --ci make the same commands work in pipelines.

```
# Install (pre-release)
dotnet tool install -g Microsoft.Maui.Cli --prerelease

# Diagnose and auto-fix your environment
maui doctor

# Android SDK, JDK and emulators
maui android
maui android emulator

# Xcode, simulators and runtimes (macOS)
maui apple

# Every connected device, every platform
maui device list

# Machine-readable output for CI
maui doctor --json --ci
```

DevFlow

An in-app agent exposes the visual tree, taps and screenshots of your running app.

Network, profiling, logs and sensors stream live, including Blazor WebViews.

The MCP server lets an AI agent test and debug the app for you.

```
# 1. Add the in-app agent to your app
dotnet add package Microsoft.Maui.DevFlow.Agent

# 2. Inspect, tap, screenshot and profile
#    the running app from your terminal
maui devflow

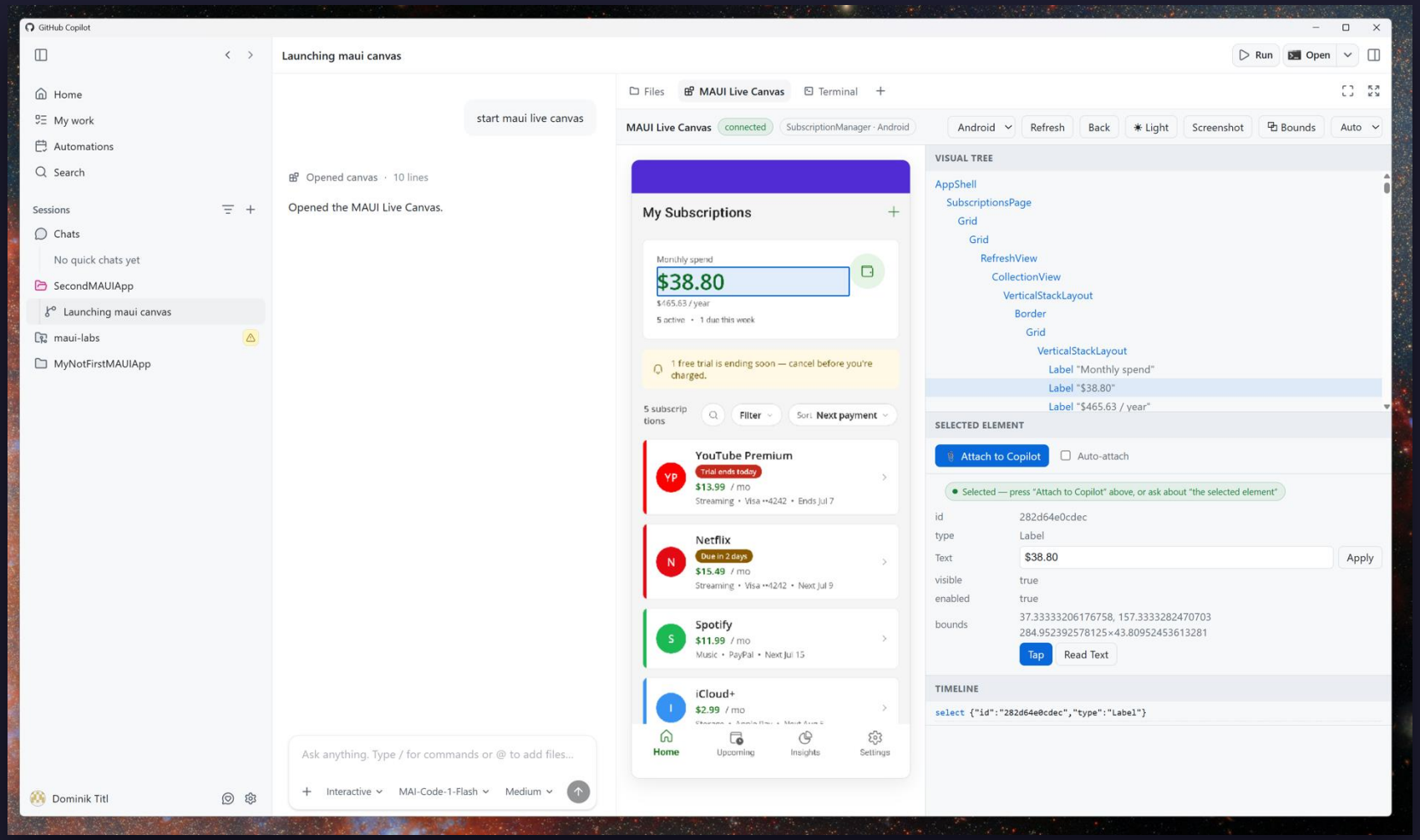
# 3. Hand the same tools to your AI agent
maui devflow mcp

# Or register the MCP server for Claude
maui ai add mcp maui-devflow --env Claude --yes

# Plain .NET Android and iOS apps, too
dotnet add package Microsoft.Maui.DevFlow.Agent.Native
```

DevFlow Inspector

Pick an element in your app, hand it to Copilot



Essentials.AI

The same IChatClient you use for cloud models, running on the device.

Apple Intelligence and NaturalLanguage do the work: no API key, no network.

Streaming, structured output, tool calling and image input are supported.

```
using Microsoft.Extensions.AI;
using Microsoft.Maui.Essentials.AI;

// On-device model: no key, no network
IChatClient chat = new AppleIntelligenceChatClient();

await foreach (var update in
    chat.GetStreamingResponseAsync("Summarize my notes"))
{
    Summary.Text += update.Text;
}

// Embeddings with Apple NaturalLanguage
var embedder = new NLEmbeddingGenerator();
```

Agent skills

maui ai init installs MAUI skills, DevFlow skills, Copilot agents and MCP config.

It detects your agent: VS Code, Copilot CLI or Claude.

--dry-run shows every change before anything is written.

```
# Preview what would be installed
maui ai init --env VsCode --dry-run

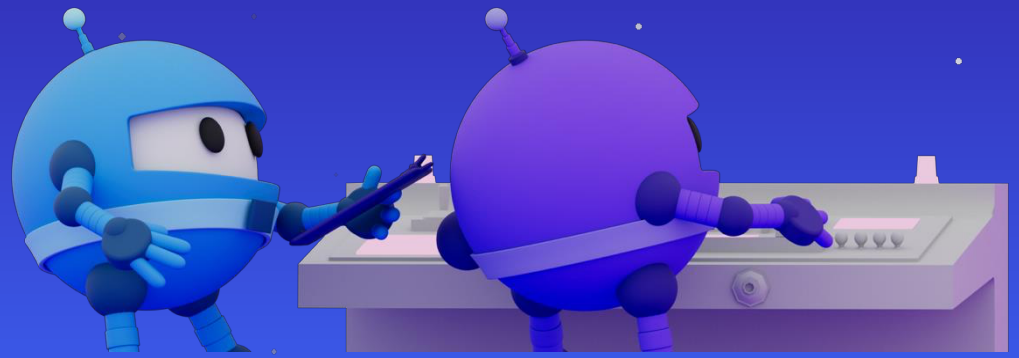
# Set up this project for AI-assisted MAUI
maui ai init --env VsCode --yes

# See what's available and what's installed
maui ai list
maui ai status

# Refresh the skills you already have
maui ai update

# Or add one skill at a time
maui ai add skill maui-devflow-debug --env Claude --yes
```

Our vision for .NET 12



Thank you

